Esercitazioni - Design Pattern: Composite

Molka Gharbaoui

Descrizione del pattern

Descrizione del pattern

- Serve a rappresentare un oggetto complesso in una struttura con relazione di contenimento.
- Scopo: Comporre oggetti in strutture che consentano di trattare i singoli elementi e la composizione in modo uniforme.
- Esempio: Le applicazioni grafiche consentono di trattare in modo uniforme sia le forme geometriche di base (linee, cerchi, …) sia gli oggetti complessi che si creano a partire da questi elementi semplici.

Descrizione del pattern

- Conseguenze:
  - I client sono semplificati perché gli oggetti semplici e quelli compositi sono trattati allo stesso modo.
  - L’aggiunta di nuovi oggetti Leaf o Composite è semplice, e questi potranno sfruttare il codice dell’applicazione Client già esistente.
  - Può rendere il sistema troppo generico. Non è possibile fare in modo che un oggetto composito contenga solo un certo tipo di oggetti.

Descrizione del pattern

- Il tipico uso di questo pattern è all'interno di frameworks grafici come AWT e Swing

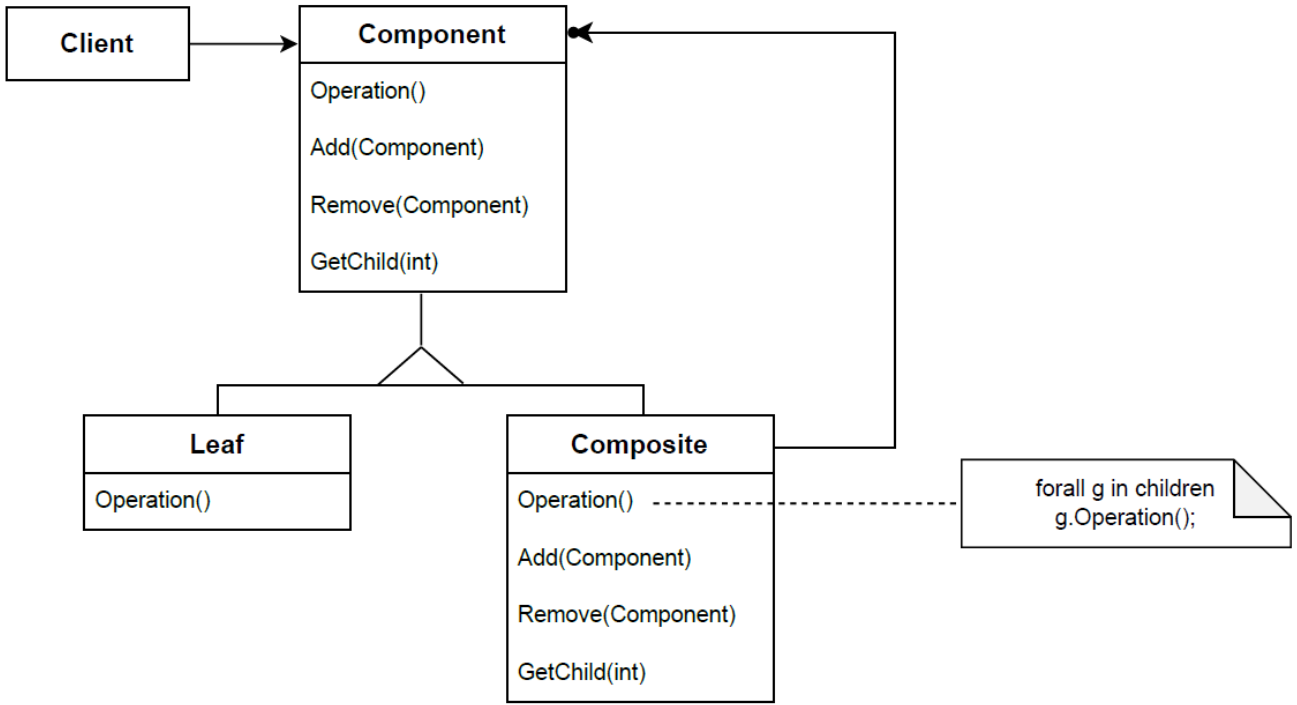
- Un’interfaccia grafica è organizzata secondo una gerarchia di contenimento: una finestra contiene dei pannelli che contengono dei bottoni, etc.

Descrizione del pattern

- Partecipanti:
  - Component: classe astratta che implementa il comportamento di default degli oggetti nella gerarchia;
  - Container: classe astratta che estende Component aggiungendo il comportamento necessario per gestire le sotto componenti;
  - ConcreteComponent1,..., ConcreteComponentN: estendono Component e sono le classi che realizzano le componenti;
  - ConcreteContainer1,..., ConcreteContainerN: estendono Container e sono le classi che realizzano i contenitori;

Struttura del pattern

Struttura del pattern



Struttura del pattern

- Consente di definire gerarchie di classi costituite da oggetti primitivi e composti:
  - Gli oggetti primitivi possono essere composti per formare oggetti più complessi, che a loro volta potranno essere composti ricorsivamente
  - Semplifica il client
  - Rende più semplice l’aggiunta di nuovi tipi di componenti

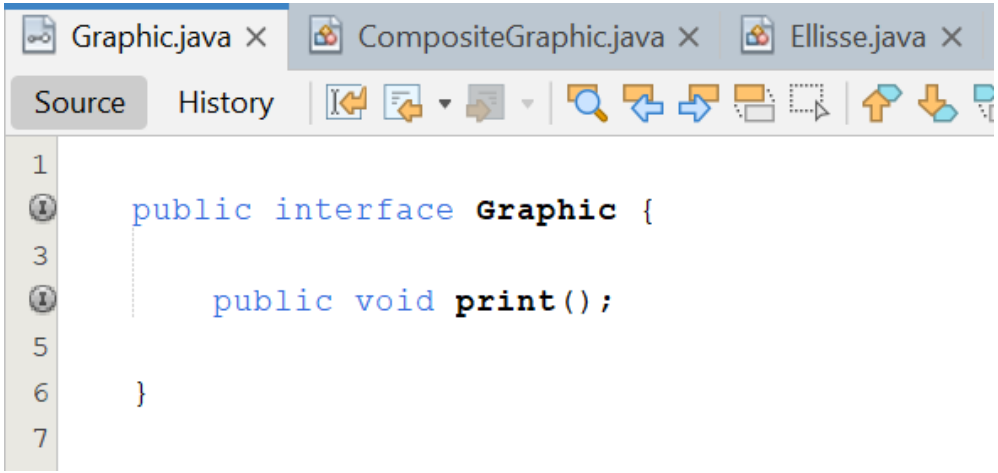
- Potrebbe essere difficile fornire un'interfaccia comune per le classi la cui funzionalità differisce troppo. Generalizzando eccessivamente l'interfaccia del componente, diventa più difficile da comprendere.

Esempi

Esempi

- Esempio 1: Implementare una classe grafica che può essere o un'ellisse o una composizione di vari ellissi. Ogni figura può essere stampata.

Esempi


Graphic.java
CompositeGraphic.java
Ellisse.java
Source
History
1
public interface Graphic {
3
public void print();
5
}
6
7

Esempi

Graphic.java | CompositeGraphic.java | Ellisse.java | Program.java

Source | History

```
1
2  import java.util.List;
3  import java.util.ArrayList;
4
5  public class CompositeGraphic implements Graphic {
6
7      private List<Graphic> mChildGraphics = new ArrayList<>();
8
9      @Override
10     public void print() {
11         for (Graphic graphic : mChildGraphics) {
12             graphic.print();
13         }
14     }
15
16     public void add(Graphic graphic) {
17         mChildGraphics.add( e: graphic);
18     }
19
20     public void remove(Graphic graphic) {
21         mChildGraphics.remove( o: graphic);
22     }
23
24 }
25
```

Esempi

Graphic.java | CompositeGraphic.java | **Ellisse.java** | Program.java

Source | History

```
1
2
3   public class Ellisse implements Graphic {
4
5       @Override
6       public void print() {
7           System.out.println( x: "Ellisse");
8       }
9
10  }
```

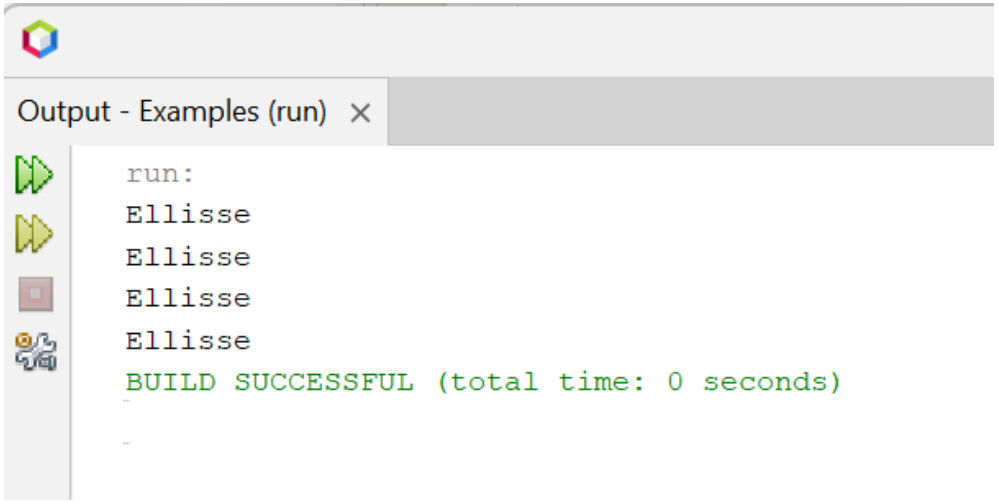
Esempi

Graphic.java | CompositeGraphic.java | Ellisse.java | Program.java

Source | History

```
1  public class Program {
2
3      public static void main(String[] args) {
4
5          Ellisse ellisse1 = new Ellisse();
6          Ellisse ellisse2 = new Ellisse();
7          Ellisse ellisse3 = new Ellisse();
8          Ellisse ellisse4 = new Ellisse();
9
10         CompositeGraphic graphic = new CompositeGraphic();
11         CompositeGraphic graphic1 = new CompositeGraphic();
12         CompositeGraphic graphic2 = new CompositeGraphic();
13
14         graphic1.add( graphic: ellisse1);
15         graphic1.add( graphic: ellisse2);
16         graphic1.add( graphic: ellisse3);
17
18         graphic2.add( graphic: ellisse4);
19
20         graphic.add( graphic: graphic1);
21         graphic.add( graphic: graphic2);
22
23         graphic.print();
24     }
25 }
26
```

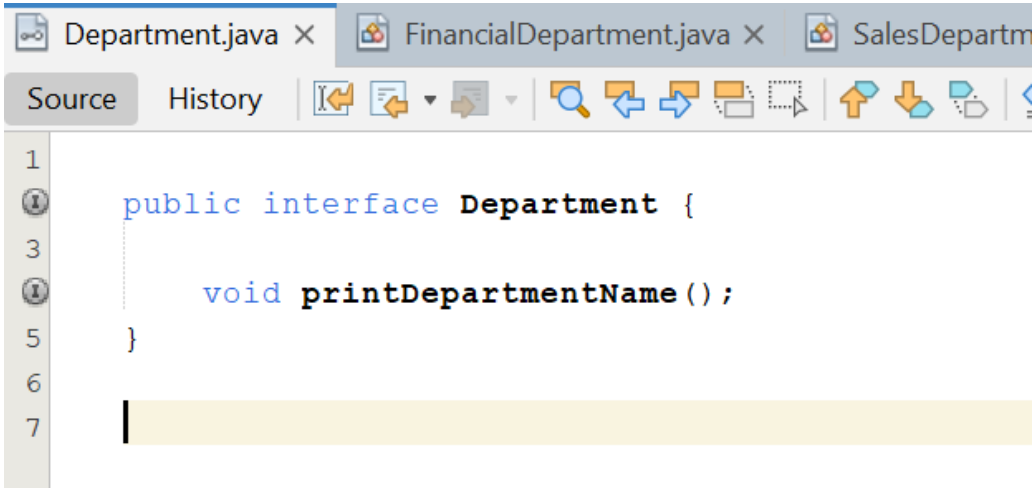
Esempi


Output - Examples (run)
run:
Ellisse
Ellisse
Ellisse
Ellisse
BUILD SUCCESSFUL (total time: 0 seconds)

Esempi

- Esempio 2: Supponiamo di voler costruire una struttura gerarchica di reparti in un'azienda usando il design pattern Composite.

Esempi


Department.java
FinancialDepartment.java
SalesDepartm
Source
History
1
public interface Department {
3
void printDepartmentName();
5
}
6
7

Esempi

Department.java | FinancialDepartment.java | SalesDepartment.java | HeadDepartment.java

Source | History

```
1
2   public class FinancialDepartment implements Department {
3
4       public FinancialDepartment(int par, String financial_department) {
5       }
6
7       private Integer id;
8       private String name;
9
10      @Override
11      public void printDepartmentName() {
12          System.out.println( x: getClass().getSimpleName());
13      }
14
15      // standard getters, setters
16  }
17
```

Esempi

Department.java | FinancialDepartment.java | SalesDepartment.java | HeadDepartment.java

Source | History

```
1
2   public class SalesDepartment implements Department {
3
4       public SalesDepartment(int par, String sales_department) {
5       }
6
7       private Integer id;
8       private String name;
9
10      @Override
11      public void printDepartmentName() {
12          System.out.println( x: getClass().getSimpleName());
13      }
14
15      // standard getters, setters
16  }
17
```

Esempi

Department.java | FinancialDepartment.java | SalesDepartment.java | HeadDepartment.java

Source | History

```java
import java.util.ArrayList;
import java.util.List;

public class HeadDepartment implements Department {

    private Integer id;
    private String name;

    private List<Department> childDepartments;

    public HeadDepartment(Integer id, String name) {
        this.id = id;
        this.name = name;
        this.childDepartments = new ArrayList<>();
    }

    public void printDepartmentName() {
        childDepartments.forEach(Department::printDepartmentName);
    }

    public void addDepartment(Department department) {
        childDepartments.add( e: department);
    }

    public void removeDepartment(Department department) {
        childDepartments.remove( o: department);
    }
}
```

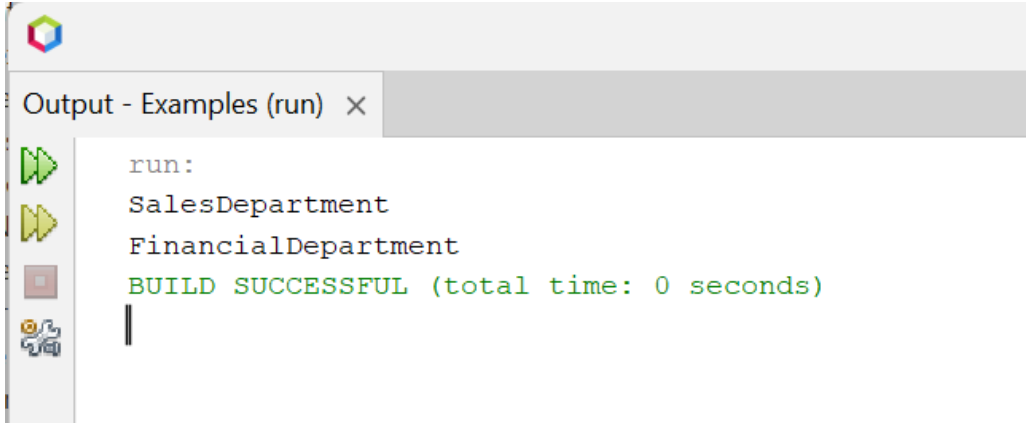
Esempi

Department.java | FinancialDepartment.java | SalesDepartment.java | HeadDepartment.java

Source | History

```
1
2   public class CompositeDemo {
3
4       public static void main(String args[]) {
5           Department salesDepartment = new SalesDepartment(
6                   par: 1,  sales_department: "Sales department");
7           Department financialDepartment = new FinancialDepartment(
8                   par: 2,  financial_department: "Financial department");
9
10          HeadDepartment headDepartment = new HeadDepartment(
11            id: 3,  name: "Head department");
12
13          headDepartment.addDepartment( department: salesDepartment);
14          headDepartment.addDepartment( department: financialDepartment);
15
16          headDepartment.printDepartmentName();
17      }
18  }
19
20
```

Esempi


Output - Examples (run)
run:
SalesDepartment
FinancialDepartment
BUILD SUCCESSFUL (total time: 0 seconds)

Esempi

- Esempio 3: Costruire una struttura gerarchica che rappresenta un carrello di prodotti (e.g., libri, eBook) usando il design pattern Composite.

Esempi

Prodotto.java

Source | History

```
1
    public interface Prodotto {
3
        public void print();
5
            public float getPrezzo();
7
            public int getPeso();
9
            public void add(Prodotto p);
11
            public void remove(Prodotto p);
13  }
```

Esempi

Prodotto.java × Libro.java ×

Source History

```java
public class Libro implements Prodotto {
        private String titolo;
        private float prezzo;
        private int peso;

        public Libro(String titol, float prez, int pes) {
                titolo = titol;
                prezzo = prez;
                peso = pes;}

        @Override
        public void print() {
                System.out.println("Libro: " + titolo + "\tPrezzo: " + getPrezzo());}

        @Override
        public float getPrezzo() {
                return prezzo;}

        @Override
        public int getPeso() {
                return peso;}

        @Override
        public void add(Prodotto p) {}

        @Override
        public void remove(Prodotto p) {}
}
```

Esempi

Prodotto.java | Libro.java | EBook.java

Source | History

```
1  public class EBook implements Prodotto {
2          private String titolo;
3          private float prezzo;
4
5          public EBook(String titol, float prez) {
6                  titolo = titol;
7                  prezzo = prez;}
8
9          @Override
10         public void print() {
11                 System.out.println("EBook: " + titolo + "\tPrezzo: " + getPrezzo());}
12
13         @Override
14         public float getPrezzo() {
15                 return (prezzo * (1 - percSconto() / 100));}
16
17         @Override
18         public int getPeso() {
19                 return 0;}
20
21         private float percSconto() {
22                 return 15;}
23
24         @Override
25         public void add(Prodotto p) {}
26
27         @Override
28         public void remove(Prodotto p) {}
```

Esempi

Prodotto.java | Libro.java | EBook.java | Carrello.java

Source | History

```
1  import java.util.ArrayList;
2  import java.util.List;
3  public class Carrello implements Prodotto {
4          private List< Prodotto > nestedElem = new ArrayList< >();
5
6          @Override
7          public void print() {
8                  System.out.println( x: "Carrello ------- ------- ---");
9                  for (Prodotto res : nestedElem)
10                         res.print();
11                 System.out.println( x: "-----------------  ------- ---");
12                 System.out.println("---------------- Prezzo totale: " + getPrezzo());}
13         @Override
14         public float getPrezzo() {
15                 return nestedElem.stream().map(e -> e.getPrezzo()).reduce( identity: 0f, Float::sum);}
16
17         @Override
18         public int getPeso() {
19                 return nestedElem.stream().map(e -> e.getPeso()).reduce( identity: 0, Integer::sum);}
20         @Override
21         public void add(Prodotto p) {
22                 System.out.println( x: "Carrello: add()");
23                 nestedElem.add( e: p);}
24
25         @Override
26         public void remove(Prodotto p) {
27                 nestedElem.remove( o: p);}}
```

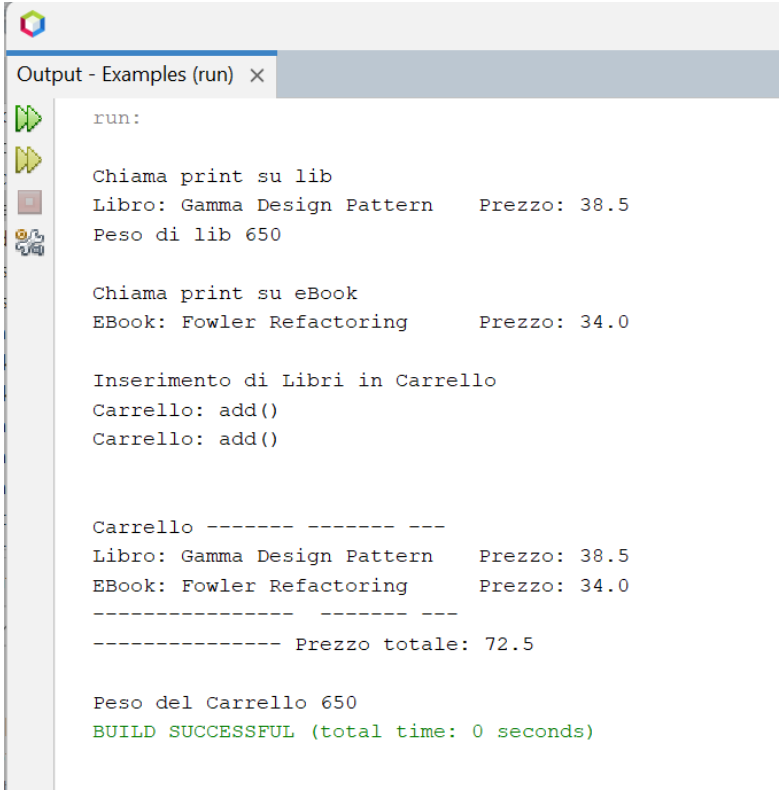
Esempi

MainCart.java

Source | History

```
1  public class MainCart {
2
3      private static final Prodotto cart = new Carrello();
4      private static final Prodotto lib = new Libro(
5              titol: "Gamma Design Pattern",  prez: 38.5f,  pes: 650);;
6      private static final Prodotto eb = new EBook(
7              titol: "Fowler Refactoring",  prez: 40.0f);;
8
9      public static void main(final String[] args) {
10         System.out.println( x: "\nChiama print su lib");
11         lib.print();
12         System.out.println("Peso di lib " + lib.getPeso());
13
14         System.out.println( x: "\nChiama print su eBook");
15         eb.print();
16
17         System.out.println( x: "\nInserimento di Libri in Carrello");
18         cart.add( p: lib);
19         cart.add( p: eb);
20
21         System.out.println( x: "\n");
22         cart.print();
23         System.out.println("\nPeso del Carrello " + cart.getPeso());
24     }}
25
```

Esempi


Output - Examples (run)
run:
Chiama print su lib
Libro: Gamma Design Pattern Prezzo: 38.5
Peso di lib 650
Chiama print su eBook
EBook: Fowler Refactoring Prezzo: 34.0
Inserimento di Libri in Carrello
Carrello: add()
Carrello: add()
Carrello ------- ------- ---
Libro: Gamma Design Pattern Prezzo: 38.5
EBook: Fowler Refactoring Prezzo: 34.0
---------- ------- ---
---------- Prezzo totale: 72.5
Peso del Carrello 650
BUILD SUCCESSFUL (total time: 0 seconds)